

AI 智能教师 —— LangChain 项目完整教学文档

适用对象：未接触过 LangChain，但想从零学起的新人。

项目简介：一个基于 LangChain + Streamlit + DeepSeek 的「高中 9 科 AI 教师」聊天应用。

前置知识：Python 基础（函数、类、文件读写）、了解什么是 LLM（大语言模型）。

目录

1. 项目概述
2. 项目目录结构
3. LangChain 项目落地流程总览
4. 模块一：LLM 模型封装 —— `core/llm.py`
5. 模块二：提示词库管理 —— `prompts/`
6. 模块三：提示词构建 —— `core/prompt_builder.py`
7. 模块四：记忆管理 —— `core/memory.py`
8. 模块五：会话管理 —— `core/session_manager.py`
9. 模块六：RAG 检索（占位） —— `core/rag.py`
10. 模块七：主应用 —— `ai_teacher_app.py`
11. 核心技术点深入讲解
12. 数据流全景图
13. 如何运行与调试

1. 项目概述

1.1 这是什么项目？

「AI 智能教师」是一个基于 LangChain 框架构建的**智能辅导聊天机器人**。用户可以在界面中选择：

- **科目**（语文、数学、英语、物理、化学、生物、政治、历史、地理 共 9 科）
- **性别**（男/女）
- **性格**（如"温柔耐心、擅长用生活例子讲解"）

然后 AI 就会按照对应的人设和教学风格，以高中老师的身份辅导学生。

1.2 技术栈一览

层级	使用的技术	作用
界面	Streamlit	提供 Web 聊天界面
编排框架	LangChain 1.x	统一管理提示词、模型、记忆
大模型	DeepSeek V4 Flash	通过 OpenAI 兼容接口调用
模型封装	langchain-openai 的 ChatOpenAI	用 OpenAI 的 SDK 封装 DeepSeek
提示词	JSON 文件 + PromptManager 类	数据与逻辑分离管理提示词
记忆	RunnableWithMessageHistory + 自定义 JsonChatHistory	会话持久化到本地 JSON 文件
RAG	待实现（Phase 3）	检索教材/讲义，让 AI 照资料讲

1.3 核心设计理念

整个项目遵循 LangChain 的核心设计哲学：**一切皆链（Chain）**。把提示词、模型、记忆像搭积木一样串起来，用 | 管道符连接：

提示词模板 | 大模型 | 记忆包装

每个组件都可以独立替换。想换模型？只改 llm.py。想加新科目？只改 JSON 文件。这就是 LangChain 的模块化优势。

2. 项目目录结构

AI_Teacher/	
├─ ai_teacher_app.py	# 🚀 主入口: Streamlit UI 界面
├─ core/	# 💬 核心逻辑包
│ └─ __init__.py	# 包标识文件
│ └─ llm.py	# ① 封装 DeepSeek 模型
│ └─ prompt_builder.py	# ② 提示词 → ChatPromptTemplate
│ └─ memory.py	# ③ 会话记忆 (JSON 持久化)
│ └─ session_manager.py	# ④ 会话 CRUD 管理
│ └─ rag.py	# ⑤ RAG 检索 (占位, 待实现)
├─ prompts/	# 📖 提示词库
│ └─ prompt_manager.py	# PromptManager 类 (逻辑)
│ └─ subject_prompts.json	# 9 科提示词数据 (数据)
├─ data/	# 📁 RAG 知识库目录 (空)
│ └─ .gitkeep	
├─ sessions/	# 💾 会话持久化目录
│ └─ 2026-09-10_20-08-13_3bed07b9.json	# 会话消息文件
│ └─ 2026-09-10_20-08-13_3bed07b9.meta.json	# 会话设置文件
└─ AI智能教师_完整教学文档.md	# 📖 本文档

3. LangChain 项目落地流程总览

在深入每个模块之前，先理解一个 LangChain 项目从零到一的完整落地流程：

第一步：搭环境
├─ 安装 LangChain 相关包
├─ 配置 API Key (环境变量)
└─ 确认模型名和接口地址
第二步：封装模型 (llm.py)
├─ 选一个 LLM 提供商 (本项目用 DeepSeek)
├─ 用 ChatOpenAI 封装 (因为 DeepSeek 兼容 OpenAI 协议)
└─ 统一所有参数 (温度、流式等)
第三步：准备提示词 (prompts/)
├─ 把提示词文案放进 JSON 文件 (数据与逻辑分离)
├─ 写一个 Manager 类来读取和拼接
└─ 设计提示词结构 (人设、守则、学科专属等)
第四步：构建提示词模板 (prompt_builder.py)
├─ 把文本提示词包装成 ChatPromptTemplate
├─ 加入 history 占位符 (为记忆预留位置)
└─ 加入 {input} 占位符 (接收用户输入)
第五步：实现记忆 (memory.py)
├─ 继承 BaseChatMessageHistory

- └─ 实现 `messages`、`add_messages`、`clear`
- └─ 持久化到本地 `JSON` 文件

第六步：管理会话（`session_manager.py`）

- └─ 新建、切换、删除、清空会话
- └─ 持久化会话设置（科目、性别、性格）

第七步：串起主应用（`ai_teacher_app.py`）

- └─ 用 `|` 管道符把 `prompt` 和 `llm` 串成链
- └─ 用 `RunnableWithMessageHistory` 包装链（加入记忆）
- └─ 用 `Streamlit` 渲染 `UI`
- └─ 实现流式输出（打字机效果）

第八步：RAG 增强（`rag.py` — 待实现）

- └─ 加载教材/讲义文档
- └─ 切片、向量化、入库
- └─ 检索后注入提示词

下面我们按这个流程，逐个模块深入讲解。

4. 模块一：LLM 模型封装 —— `core/llm.py`

新手先理解：LLM 就是大语言模型（Large Language Model）。这个模块的作用是统一模型的入口，以后换模型、调参数，只改这一个文件就行。

4.1 完整源码与逐行注释

```
"""
LLM 模型封装模块

统一 DeepSeek 模型的入口，用 langchain-openai 的 ChatOpenAI 封装 DeepSeek
（DeepSeek 是 OpenAI 兼容协议）。以后要换模型 / 调参数，只改这一个文件即可。

说明：
- base_url 指向 DeepSeek 官方接口
- api_key 从环境变量 DEEPSEEK_API_KEY 读取，不硬编码
- extra_body 关闭 thinking（deepseek-v4-flash 默认带推理，关掉后走普通对话，
  流式输出时不会出现 delta.content 为 None 的思考阶段，解析更简单）
"""
# — 以上是模块文档字符串（docstring），描述模块的用途和设计思路 —

import os
# os 是 Python 标准库，用于读取操作系统环境变量。
# 这里用它来读取 DEEPSEEK_API_KEY，而不是把密钥写在代码里。
```

```

from langchain_openai import ChatOpenAI
# ChatOpenAI 是 langchain-openai 包提供的类。
# 它的作用是：用 OpenAI 的通信协议（HTTP 请求格式）去调用大模型。
# 因为 DeepSeek 的 API 格式和 OpenAI 完全兼容，所以可以直接用这个类，
# 只需要把 base_url 从 OpenAI 的地址改成 DeepSeek 的地址即可。

# — 模型配置常量 —

MODEL_NAME = "deepseek-v4-flash"
# 模型名称。这是 DeepSeek 在 2026 年 7 月后使用的 V4 系列模型名。
# 旧名称 deepseek-chat / deepseek-reasoner 已于 2026-07-24 停用。
# 参数解释：
#   - "deepseek-v4-flash": 快速版，适合日常对话，响应快
#   - "deepseek-v4-pro": 专业版，推理能力更强，但更慢更贵

BASE_URL = "https://api.deepseek.com"
# DeepSeek 的 API 接口地址。
# 这是 OpenAI 兼容接口的地址，ChatOpenAI 会把请求发到这里。
# 参数解释：
#   - 如果直接用 OpenAI，这个值会是 "https://api.openai.com/v1"
#   - 改成 DeepSeek 的地址后，ChatOpenAI 就会把请求发给 DeepSeek

def get_llm() -> ChatOpenAI:
    """返回一个封装好 DeepSeek 的 ChatOpenAI 实例。

    Returns:
        ChatOpenAI: 可直接用于 prompt | llm 链的模型对象。
    """
    # ↑ 函数文档字符串，描述返回值类型和用途

    # — 步骤 1: 读取 API Key —
    api_key = os.environ.get("DEEPSEEK_API_KEY")
    # os.environ 是一个字典，包含所有系统环境变量。
    # .get("DEEPSEEK_API_KEY") 尝试读取名为 DEEPSEEK_API_KEY 的环境变量。
    # 如果没设置这个变量，返回 None。
    # 为什么用环境变量？— 安全！密钥不写进代码，不会不小心提交到 Git。

    if not api_key:
        raise EnvironmentError(
            "未找到 DEEPSEEK_API_KEY，请在环境变量或 .env 文件中配置 DeepSeek API Key。"
        )
    # 如果 api_key 为空（None 或 ""），抛出明确的错误提示。
    # EnvironmentError 是 Python 内置异常，表示环境配置有问题。

    # — 步骤 2: 创建并返回 ChatOpenAI 实例 —
    return ChatOpenAI(
        model=MODEL_NAME,
        # model 参数：指定要使用的模型名称。
        # 值：上面定义的 "deepseek-v4-flash"
        # 这个参数告诉 DeepSeek："用 V4 Flash 这个模型来回答问题"

        api_key=api_key,
        # api_key 参数：你的 DeepSeek API 密钥。
        # 相当于你的"身份凭证"，DeepSeek 用它来识别你是谁、扣你的费用。
        # 值：从环境变量读取的密钥字符串，格式通常是 "sk-xxxxxxxxxxxxxxxx"

        base_url=BASE_URL,

```

```
# base_url 参数: API 接口的基础地址。
# ChatOpenAI 默认发请求到 OpenAI 的服务器。
# 设置 base_url="https://api.deepseek.com" 后, 请求就发到 DeepSeek。
# 这就是"OpenAI 兼容协议"的妙用: 换一个地址就能换一个模型提供商。

temperature=0.7,
# temperature 参数: 生成温度, 控制回答的随机性/创造性。
# 取值范围: 0.0 ~ 2.0 (通常用 0.0 ~ 1.0)
#   - 0.0: 最确定, 每次回答几乎一样 (适合数学、编程等需要精确的场景)
#   - 0.7: 比较自然, 有一定的变化和创造性 (适合教学对话)
#   - 1.0+: 非常发散, 可能产生意想不到的内容 (适合创意写作)
# 这里选 0.7: 教学场景需要在准确和自然之间取得平衡。

streaming=True,
# streaming 参数: 是否开启流式输出。
#   - True: 模型边生成边返回, 可以逐字显示 (打字机效果)
#   - False: 等模型全部生成完再一次性返回
# 这里选 True: 后续用 .stream() 方法实现打字机效果, 用户体验更好。

extra_body={"thinking": {"type": "disabled"}},
# extra_body 参数: 额外的请求体内容, 会附加到发给 DeepSeek 的 HTTP 请求中。
# 这里的作用是关闭 DeepSeek V4 的"思考模式"。
# 解释:
#   - deepseek-v4-flash 默认会先"思考"再回答 (类似 o1 的推理过程)
#   - 思考阶段流式输出时, delta.content 会是 None (因为没有正文内容)
#   - 关闭后, 模型直接输出正文, 流式解析更简单
#   - {"thinking": {"type": "disabled"}} 是 DeepSeek 特有的参数
)
```

4.2 新手常见疑问

Q1: 为什么用 **ChatOpenAI** 而不是 DeepSeek 专用的类?

A: 因为 DeepSeek 的 API 和 OpenAI 的 API 格式完全一样 (这叫"OpenAI 兼容协议")。LangChain 的 **ChatOpenAI** 本质上是发 HTTP 请求的客户端, 只要把请求地址从 OpenAI 改成 DeepSeek, 它就能正常工作。这就像你的手机支持 Type-C 充电, 不管插的是小米还是华为的充电器都能充。

Q2: **temperature** 到底怎么选?

A: 简单记:

- 0.0 ~ 0.3: 严谨场景 (数学证明、代码生成)
- 0.5 ~ 0.7: 日常对话、教学 (本项目用的范围)
- 0.8 ~ 1.0: 创意写作、头脑风暴

Q3: **streaming=True** 和 **.stream()** 有什么区别?

A: `streaming=True` 是告诉模型"请支持流式输出"。`.stream()` 是调用时实际使用流式方式接收数据。两者缺一不可。

5. 模块二：提示词库管理 —— `prompts/`

新手先理解：提示词（Prompt）就是你跟 AI 说的话。这个模块把提示词分成**数据**（JSON 文件）和**逻辑**（Python 类），改内容只改 JSON，不用动代码。

5.1 数据层： `prompts/subject_prompts.json`

这个 JSON 文件是整个项目的"知识库"，定义了 AI 教师的人设和教学规则。它分为两大部分：

```
{
  "_shared": {
    // — 第一部分：所有科目共享的通用内容 —

    "persona_template": "你是学生的{subject}老师（性别：{gender}，教学性格：{personality}）。请完全代入这个角色，像一位真实的老师那样辅导学生。",
    // persona_template: 人设模板，包含三个占位符
    //   - {subject}: 科目名，运行时会被替换为"数学"、"语文"等
    //   - {gender}: 性别，运行时会被替换为"男"或"女"
    //   - {personality}: 性格描述，运行时会被替换为用户输入的文字
    // 这是 Python 的 str.format() 语法，{变量名} 就是占位符

    "general_rules": [
      "先弄清学生的基础和具体困惑，针对性讲解，不要自说自话。",
      "每次聚焦一个知识点，讲透一个再讲下一个，循序渐进。",
      "引导式教学：先给思路和线索，让学生自己尝试，不直接甩答案。",
      "多用简单例子、生活实例或类比，把抽象概念讲具体。",
      "适时提醒易错点、易混点，帮学生避开常见坑。",
      "多鼓励、肯定学生，语气耐心，不打击信心。",
      "回答结构清晰，适当分点分步骤，但避免堆砌术语。"
    ],
    // general_rules: 通用教学守则，共 7 条，所有科目老师都必须遵守
    // 这是一个数组（列表），每条是一条规则，运行时编号输出

    "output_style": [
      "用与「教学性格」相符的口吻和学生交流（如耐心、幽默、严厉等，由用户设定）。",
      "每轮回复聚焦当前问题，简短但把关键点讲清楚。",
      "学生问作业时，先带思路再逐步推导，最终落到「让学生自己会做」。",
      "需要时可以画图、列式子、分步骤，帮助理解。"
    ],
    // output_style: 输出风格要求，共 4 条
    // 这告诉 AI 回答时应该用什么语气和格式
  },

  "subjects": {
```

```
"数学": {
    // 每个科目有 4 个字段，结构完全相同

    "focus": "数学培养逻辑推理、抽象思维与运算求解能力。",
    // focus: 学科定位，一句话概括这个学科的核心素养

    "teaching_points": [
        "函数与导数",
        "三角函数与解三角形",
        "数列",
        "立体几何与解析几何",
        "概率与统计",
        "不等式"
    ],
    // teaching_points: 教学侧重点，列出这个学科的主要知识板块

    "question_types": [
        "选择题",
        "填空题",
        "解答题（证明/计算）",
        "压轴题"
    ],
    // question_types: 常见题型，告诉 AI 学生会问什么类型的题目

    "method_guide": [
        "先讲清概念与公式的来龙去脉，不让学生死记硬背。",
        "解题遵循「审题→分析→列式→计算→检验」五步。",
        "典型例题精讲，归纳一类题的通法。",
        "强调书写步骤规范与易错点（符号、定义域、单位）。",
        "抽象问题尽量画图，数形结合。"
    ]
    // method_guide: 本学科讲解方法，针对这个学科的特殊教学方法
},

// 语文、英语、物理、化学、生物、政治、历史、地理 结构完全相同
// 每个科目都有 focus、teaching_points、question_types、method_guide 四个字段
}
```

设计优势：新增科目只需在 JSON 里加一个条目，Python 代码完全不用改。这就是"数据与逻辑分离"。

5.2 逻辑层： `prompts/prompt_manager.py`

```
"""
提示词库管理模块

负责从 subject_prompts.json 中读取 9 个高中科目的提示词，
根据「科目 + 性别 + 性格」三个变量，组装出完整的系统提示词。

设计思路：
```

- 提示词文案全部放在 JSON 文件里（数据），本类只负责读取与拼接（逻辑）。
- 新增/修改科目，只需改 JSON，不用改这里的代码。
- 三个变量：科目(subject)、性别(gender)、性格(personality)。

```
"""
```

```
import json
```

```
# json: Python 标准库，用于读写 JSON 文件
```

```
# 这里用它来加载 subject_prompts.json
```

```
import os
```

```
# os: Python 标准库，用于处理文件路径
```

```
# 这里用它来拼接 JSON 文件所在的目录路径
```

```
class PromptManager:
```

```
    """提示词库管理器：加载 JSON 并按需生成系统提示词。"""
```

```
    def __init__(self, json_path=None):
```

```
        """
```

```
        初始化：加载提示词 JSON 文件。
```

```
        Args:
```

```
            json_path (str, optional): JSON 文件路径。默认使用本文件同目录下的  
                subject_prompts.json。
```

```
        """
```

```
        # — 参数说明 —
```

```
        # json_path: JSON 文件的路径，可以不传（默认 None）
```

```
        # - 传了：用指定的路径
```

```
        # - 不传：自动找本文件同目录下的 subject_prompts.json
```

```
    if json_path is None:
```

```
        # — 自动拼接默认路径 —
```

```
        json_path = os.path.join(  
            os.path.dirname(__file__), # 当前文件所在目录 (prompts/)  
            "subject_prompts.json"    # JSON 文件名  
        )
```

```
        # os.path.dirname(__file__): 获取当前 .py 文件所在的目录路径
```

```
        # - __file__ 是 Python 内置变量，值为当前文件的路径
```

```
        # - os.path.dirname() 取路径的目录部分
```

```
        # os.path.join(): 安全地拼接路径（自动处理 / 和 \ 的差异）
```

```
        # 最终得到类似：D:\...\AI_Teacher\prompts\subject_prompts.json
```

```
    # — 打开并读取 JSON 文件 —
```

```
    with open(json_path, "r", encoding="utf-8") as f:
```

```
        # open() 参数:
```

```
        # - json_path: 文件路径
```

```
        # - "r": 只读模式 (read)
```

```
        # - encoding="utf-8": 指定编码为 UTF-8（确保中文正常读取）
```

```
        # with 语句：自动管理文件关闭，即使出错也会关闭
```

```
        self._data = json.load(f)
```

```
        # json.load(f): 把 JSON 文件内容解析成 Python 字典
```

```
        # 此时 self._data 就是整个 JSON 文件的内容
```

```
        # 以 _ 开头的变量名是 Python 约定：表示"私有属性"，外部不要直接访问
```

```
    # — 拆分成两部分，方便后续使用 —
```

```
    self._shared = self._data["_shared"]
```

```
    # _shared: 通用内容字典
```

```
    # 包含 persona_template、general_rules、output_style 三个字段
```

```
self._subjects = self._data["subjects"]
# _subjects: 各科目专属内容字典
# 键是科目名（"数学"、"语文"...），值是该科目的四个字段
```

```
def list_subjects(self):
    """返回所有可用科目名列表，例如 ["语文", "数学", ...]"""
    return list(self._subjects.keys())
# _subjects.keys() 返回字典的所有键: dict_keys(['语文', '数学', ...])
# list() 把它转成普通的列表: ['语文', '数学', ...]
# 这个列表会传给 Streamlit 的下拉框组件
```

```
def has_subject(self, subject):
    """判断某个科目是否存在。"""
    return subject in self._subjects
# in 运算符检查字典是否包含某个键
# 返回 True 或 False
```

```
def build_system_prompt(self, subject, gender, personality):
    """
    根据三个变量生成完整系统提示词。
```

Args:

subject (str): 科目，如 "数学"。
gender (str): 性别，如 "男" / "女"。
personality (str): 性格/教学风格，如 "温柔耐心、擅长用生活例子讲解"。

Returns:

str: 组装好的系统提示词（可直接作为 system message 的 content）。

"""

— 步骤 1: 生成人设文本 —

```
persona = self._shared["persona_template"].format(
    subject=subject, gender=gender, personality=personality
)
```

.format() 是 Python 字符串方法，用来替换模板中的占位符。

模板: "你是学生的{subject}老师（性别: {gender}, 教学性格: {personality}）..."

替换后: "你是学生的数学老师（性别: 女, 教学性格: 温柔耐心、擅长用生活例子讲

解）..."

这是 Python 字符串格式化的两种方式之一（另一种是 f-string）

— 步骤 2: 逐块拼接 —

```
lines = [persona, ""]
```

用列表收集所有行，最后用换行符连接

persona 是第一行，"" 是空行（分隔）

— 2.1 通用教学守则 —

```
lines.append("【教学守则 · 所有科目通用】")
```

添加小标题

```
for i, rule in enumerate(self._shared["general_rules"], 1):
```

enumerate(列表, 起始编号):

- i 是编号（从 1 开始）

- rule 是当前规则文本

```
lines.append(f"{i}. {rule}")
```

格式: 1. 先弄清学生的基础和具体困惑...

```
lines.append("")
```

空行分隔

— 2.2 学科专属内容 —

```
subj = self._subjects.get(subject)
```

.get(subject) 安全地获取科目数据

```

# 如果科目不存在, 返回 None (不会报错)
# 等价于 self._subjects[subject] 但更安全

if subj:
    # 科目存在时, 输出完整的学科专属内容
    lines.append(f"【学科定位】{subject}: {subj['focus']}")
    lines.append("")

    lines.append("【教学侧重点】")
    for point in subj["teaching_points"]:
        lines.append(f"- {point}")
    lines.append("")

    lines.append("【常见题型】")
    for q in subj["question_types"]:
        lines.append(f"- {q}")
    lines.append("")

    lines.append("【本学科讲解方法】")
    for i, method in enumerate(subj["method_guide"], 1):
        lines.append(f"{i}. {method}")
    lines.append("")
else:
    # 科目不存在时, 给一个通用的退路提示
    lines.append("【学科定位】请以一位负责的老师身份, 就学生提出的问题给出专业、清晰的辅导。")
    lines.append("")

# — 2.3 输出风格 —
lines.append("【输出风格】")
for style in self._shared["output_style"]:
    lines.append(f"- {style}")

# — 步骤 3: 把所有行用换行符连接成一段完整文本 —
return "\n".join(lines)
# "\n".join(列表): 把列表中的每个元素用换行符连接
# 最终返回的是一段包含所有提示词内容的多行文本

# — 自测代码 —
if __name__ == "__main__":
    # 这段代码只在直接运行这个文件时执行 (python prompt_manager.py)
    # 被 import 导入时不会执行

    pm = PromptManager()
    print("已加载科目: ", pm.list_subjects())
    print("\n" + "=" * 60)
    print(pm.build_system_prompt("数学", "女", "温柔耐心、擅长用生活例子讲解"))
    # 打印一个示例, 验证提示词拼接是否正确

```

5.3 提示词的七块结构

最终生成的系统提示词按以下顺序排列：

1. 人设
 - └─ "你是学生的数学老师（性别：女，教学性格：温柔耐心...）"
2. 通用教学守则（7条）
 - └─ 1. 先弄清学生的基础和具体困惑...
 - └─ 2. 每次聚焦一个知识点...
 - └─ ...
3. 学科定位
 - └─ 数学：数学培养逻辑推理、抽象思维与运算求解能力。
4. 教学侧重点
 - └─ - 函数与导数
 - └─ - 三角函数与解三角形
 - └─ ...
5. 常见题型
 - └─ - 选择题
 - └─ - 填空题
 - └─ ...
6. 本学科讲解方法
 - └─ 1. 先讲清概念与公式的来龙去脉...
 - └─ 2. 解题遵循「审题→分析→列式→计算→检验」五步...
 - └─ ...
7. 输出风格
 - └─ - 用与「教学性格」相符的口吻...
 - └─ - 每轮回复聚焦当前问题...
 - └─ ...

6. 模块三：提示词构建 —— `core/prompt_builder.py`

新手先理解：上一步产出的是一段"纯文本"提示词。但 LangChain 需要的是结构化的"模板"（ChatPromptTemplate）。这个模块就是做"文本 → 模板"的转换。

6.1 完整源码与逐行注释

```
"""
```

```
提示词构建模块
```

```
把 prompts/ 下 PromptManager 生成的中文系统提示词，包装成 LangChain 的  
ChatPromptTemplate，供 prompt | llm 链使用。
```

```
模板结构（三段）：
```

```

system    — 由「科目 + 性别 + 性格」生成的人设/教学提示词
history   — 历史消息，由 RunnableWithMessageHistory 自动填充
human     — 用户当前输入 {input}
"""

import os
import sys

from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
# ChatPromptTemplate: LangChain 的聊天提示词模板类
# - 它把提示词组织成"消息列表"的形式 (system、human、ai 等角色)
# - 就像 ChatGPT 背后的消息格式: [{"role": "system", "content": "..."}, ...]
# MessagesPlaceholder: 消息占位符
# - 在模板中预留一个位置，让后续组件（如记忆组件）把一组消息填进去
# - 这里用来放历史对话记录

# — 路径处理：确保能导入 prompts 包 —
# 因为 prompts/ 目录不在 core/ 的子目录里，需要把项目根目录加入 Python 搜索路径
_PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
# 逐层拆解：
# __file__                → core/prompt_builder.py 的路径
# os.path.abspath(__file__) → 转成绝对路径
# os.path.dirname(...)    → 向上取目录 → core/ 目录
# os.path.dirname(...) 再来一次 → 再向上取目录 → AI_Teacher/ 目录（项目根）
if _PROJECT_ROOT not in sys.path:
    sys.path.insert(0, _PROJECT_ROOT)
# sys.path 是 Python 导入模块时的搜索路径列表
# 把项目根目录加进去，就能 import prompts 包了

from prompts.prompt_manager import PromptManager # noqa: E402
# noqa: E402 告诉代码检查工具：这个 import 不在文件顶部是有意为之的
# 因为必须先修改 sys.path，才能导入 prompts 包

def build_chat_prompt(subject, gender, personality):
    """根据三个变量，返回一个带 history 占位符的 ChatPromptTemplate。

    Args:
        subject (str): 科目，如 "数学"。
        gender (str): 性别，如 "男" / "女"。
        personality (str): 性格/教学风格，如 "温柔耐心、擅长用生活例子讲解"。

    Returns:
        ChatPromptTemplate: 含 system / history / human 三段的提示词模板。
    """
    # — 步骤 1: 获取纯文本的系统提示词 —
    pm = PromptManager()
    # 创建 PromptManager 实例，它会自动加载 JSON 文件
    system_text = pm.build_system_prompt(subject, gender, personality)
    # 调用 build_system_prompt 生成完整的系统提示词文本
    # system_text 是一段多行文本，包含人设、守则、学科内容等

    # — 步骤 2: 包装成 ChatPromptTemplate —
    return ChatPromptTemplate.from_messages(
        # from_messages() 是一个类方法，根据消息列表创建模板
        # 参数是一个列表，每个元素是 (角色, 内容) 的元组
        [
            ("system", system_text),

```

```

# 第一条消息: system 角色
# - "system" 是角色名, 表示这是系统指令
# - system_text 是系统提示词的内容 (就是上面生成的那段文本)
# - 在发给模型的请求中, 它会变成:
#   {"role": "system", "content": "你是学生的数学老师..."}

MessagesPlaceholder(variable_name="history"),
# 第二条消息: 历史占位符
# - 这不是一条具体的消息, 而是一个"插槽"
# - variable_name="history" 是这个插槽的名字
# - 运行时, RunnableWithMessageHistory 会自动把历史消息填进这个位置
# - 历史消息可能是多条: 用户之前问的 + AI 之前答的

("human", "{input}"),
# 第三条消息: 用户当前输入
# - "human" 是角色名, 表示这是用户说的话
# - "{input}" 是占位符, 运行时会被替换成用户实际输入的文字
# - 注意这里用的是 {input} (带花括号), 不是纯文本
# - LangChain 的 .invoke({"input": "..."}) 或 .stream({"input":
"..."})
#   会自动把 {input} 替换成对应的值
]
)

```

6.2 为什么需要 ChatPromptTemplate?

直接用文本也能跟模型对话, 为什么要多此一举?

因为 LangChain 的链式调用需要统一接口。看这个对比:

```

# ❌ 不用 LangChain 的做法 (原始方式)
system_prompt = "你是数学老师..."
user_input = "什么是函数?"
response = model.chat(system=system_prompt, user=user_input)
# 问题: 每次都要手动传 system_prompt, 还要自己管理历史消息

# ✅ 用 LangChain 的做法 (模板化)
template = ChatPromptTemplate.from_messages([
    ("system", system_text),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
])
chain = template | model
# 优势: 模板统一了输入格式, 历史自动管理, 可以链式调用

```

6.3 三段式模板结构详解

ChatPromptTemplate

```
("system", system_text)
系统提示词（固定，每次对话都一样）
"你是学生的数学老师，性别：女..."
```

```
MessagesPlaceholder("history")
历史消息占位符（动态，由记忆组件填充）
[HumanMessage("什么是函数？"),
 AIMessage("函数是一种映射关系..."),
 HumanMessage("能举个例子吗？")]
```

```
("human", "{input}")
用户当前输入（动态，由 .invoke() 传入）
"能再详细一点吗？"
```

7. 模块四：记忆管理 —— `core/memory.py`

新手先理解：大模型本身是"无状态"的——它不记得上一轮说过什么。记忆模块就是给模型加上"记忆"，让它能记住对话历史。LangChain 1.x 提供了 `BaseChatMessageHistory` 基类，我们继承它来实现自定义的存储方式。

7.1 完整源码与逐行注释

```
"""
```

LangChain 记忆（会话持久化）模块

用自定义的 `JsonChatHistory` 实现 `BaseChatMessageHistory`：把每个会话的消息历史持久化到 `sessions/{session_id}.json`。

LangChain 1.x 的 `BaseChatMessageHistory` 约定：

- 需要实现 `messages` 属性（读）、`add_messages`（批量写）、`clear`（清空）
- `add_message`（单条）会默认委托给 `add_messages`，不用重复实现

```
"""
```

```
import json
```

```
# json: 读写 JSON 文件
```

```
import os
```

```
# os: 文件路径处理和目录创建
```

```

from datetime import datetime
# datetime: 获取当前时间，用于生成会话 ID

from uuid import uuid4
# uuid4: 生成随机 UUID（通用唯一标识符）
# 用来给会话 ID 加随机后缀，避免同一时间创建的两个会话重名

from langchain_core.chat_history import BaseChatMessageHistory
# BaseChatMessageHistory: LangChain 的记忆基类
# 继承它并实现指定方法，就能接入 LangChain 的记忆体系

from langchain_core.messages import messages_from_dict, messages_to_dict
# messages_from_dict: 把字典列表转成 LangChain 消息对象列表
# messages_to_dict: 把 LangChain 消息对象列表转成字典列表
# 这两个函数是 LangChain 官方的序列化/反序列化工具

# — 全局配置 —

_PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
# 获取项目根目录（AI_Teacher/）
# 计算过程：__file__ → 绝对路径 → 上一级(core/) → 上一级(AI_Teacher/)

SESSIONS_DIR = os.path.join(_PROJECT_ROOT, "sessions")
# 会话文件存储目录：AI_Teacher/sessions/
# os.path.join 自动处理路径分隔符（Windows 用 \，Linux/Mac 用 /）

class JsonChatHistory(BaseChatMessageHistory):
    """把消息历史持久化到本地 JSON 文件的历史存储。"""

    def __init__(self, session_id, directory=None):
        """初始化一个会话历史。

        Args:
            session_id (str): 会话 ID，对应文件 sessions/{session_id}.json。
            directory (str, optional): 存储目录，默认 SESSIONS_DIR。
        """
        self.session_id = session_id
        # 会话 ID，是整个会话的唯一标识
        # 格式如：2026-09-10_20-08-13-139187_3bed07b9

        self.directory = directory or SESSIONS_DIR
        # 存储目录，如果没传就用默认的 SESSIONS_DIR
        # "or" 运算符：如果 directory 是 None 或空字符串，就用 SESSIONS_DIR

        os.makedirs(self.directory, exist_ok=True)
        # 确保目录存在
        # - exist_ok=True: 如果目录已存在不报错
        # - 不存在则递归创建（mkdir -p 的效果）

        self._path = os.path.join(self.directory, f"{session_id}.json")
        # 拼接出完整的文件路径
        # 例如：AI_Teacher/sessions/2026-09-10_20-08-13-139187_3bed07b9.json
        # 以 _ 开头的属性名表示"私有"，外部不应直接访问

    @property
    def messages(self):
        """读取当前会话的全部消息（文件不存在时返回空列表）。

```

这是 `BaseChatMessageHistory` 要求实现的属性（`property`）。

`LangChain` 内部会调用这个属性来获取历史消息。

```
"""
```

```
if not os.path.exists(self._path):
```

```
    # 文件不存在（新会话，还没有任何消息）
```

```
    return []
```

```
    # 返回空列表，表示没有历史
```

```
try:
```

```
    with open(self._path, "r", encoding="utf-8") as f:
```

```
        # 打开 JSON 文件，只读模式
```

```
        # encoding="utf-8" 确保中文正常读取
```

```
        data = json.load(f)
```

```
        # 把 JSON 内容解析成 Python 对象（列表）
```

```
        # 格式: [{"type": "human", "content": "..."}, {"type": "ai",
```

```
"content": "..."}]
```

```
    return messages_from_dict(data)
```

```
    # messages_from_dict(): LangChain 官方函数
```

```
    # 把字典列表转成 LangChain 的消息对象列表
```

```
    # 输入: [{"type": "human", "content": "你好"}]
```

```
    # 输出: [HumanMessage(content="你好")]
```

```
    # 这样 LangChain 的其他组件就能直接使用这些消息对象了
```

```
except (json.JSONDecodeError, OSError, TypeError, KeyError) as exc:
```

```
    # 捕获可能的异常:
```

```
    # - JSONDecodeError: JSON 格式损坏
```

```
    # - OSError: 文件读取失败（权限问题等）
```

```
    # - TypeError: 数据类型不匹配
```

```
    # - KeyError: 字典缺少必要的键
```

```
    raise ValueError(
```

```
        f"会话文件损坏或无法读取: {self._path} ({exc})"
```

```
    ) from exc
```

```
    # 把这些异常统一转成 ValueError，并带上文件路径和原因
```

```
    # from exc 保留原始异常链，方便调试
```

```
def add_messages(self, messages):
```

```
    """批量追加消息并写回文件（读旧 + 追加新，再整体落盘）。
```

```
    Args:
```

```
        messages (Sequence[BaseMessage]): 要追加的消息列表。
```

```
        Sequence 是 Python 类型注解，表示"可迭代的序列"（如列表、元组）。
```

```
    """
```

```
    # — 步骤 1: 读旧消息 —
```

```
    all_messages = list(self.messages) + list(messages)
```

```
    # self.messages 调用上面的 @property，返回已有的消息列表
```

```
    # list() 确保是列表类型（以防万一）
```

```
    # + 把新旧消息合并
```

```
    # — 步骤 2: 整体写回文件 —
```

```
    with open(self._path, "w", encoding="utf-8") as f:
```

```
        # "w" 模式: 覆盖写入（清空旧内容，写新内容）
```

```
        json.dump(
```

```
            messages_to_dict(all_messages),
```

```
            # messages_to_dict(): LangChain 官方函数
```

```
            # 把消息对象列表转成字典列表（方便 JSON 序列化）
```

```
            # 输入: [HumanMessage(content="你好"), AIMessage(content="你好!")]
```

```
            # 输出: [{"type": "human", "content": "你好"},
```

```
                    {"type": "ai", "content": "你好!"}]
```

```

        f,
        # 目标文件对象

        ensure_ascii=False,
        # 关键参数! ensure_ascii=False:
        #   - True (默认): 中文会被转义成 \uXXXX 格式 (如 "\u4f60\u597d")
        #   - False: 中文直接原样输出 (如 "你好")
        # 这里必须设为 False, 否则 JSON 文件里的中文会变成乱码

        indent=2,
        # 缩进 2 个空格, 让 JSON 文件格式化输出, 方便人类阅读
    )

```

```

def clear(self):
    """清空当前会话的所有消息。"""
    with open(self._path, "w", encoding="utf-8") as f:
        json.dump([], f, ensure_ascii=False)
        # 写入一个空列表 [], 表示清空所有消息
        # 注意: 这里只清空内容, 不删除文件
        # 会话 ID 和文件本身保留

```

```

@property
def path(self):
    """返回会话文件的完整路径。"""
    return self._path

```

— 以下是辅助函数, 不是类的方法 —

```

def create_session_id():
    """仅生成 ID; 随机后缀避免快速点击或多个页面创建时重名。"""
    return f"{datetime.now():%Y-%m-%d_%H-%M-%S-%f}_{uuid4().hex[:8]}"
    # 生成格式: 年-月-日_时-分-秒-微秒_随机8位
    # 例如: 2026-09-10_20-08-13-139187_3bed07b9
    # 拆解:
    #   datetime.now()           → 当前日期时间
    #   :%Y-%m-%d_%H-%M-%S-%f   → 格式化字符串
    #   %Y = 四位年份 (2026)
    #   %m = 两位月份 (09)
    #   %d = 两位日期 (10)
    #   %H = 24小时制小时 (20)
    #   %M = 分钟 (08)
    #   %S = 秒 (13)
    #   %f = 微秒 (139187)
    #   uuid4().hex[:8]          → 随机 UUID 的前 8 位十六进制字符
    # 微秒 + 随机后缀双重保障, 几乎不可能重名

```

```

def create_session(directory=None):
    """创建空会话文件并返回 ID, 让侧边栏能立即列出新会话。"""
    while True:
        # 无限循环, 直到成功创建不重名的文件
        session_id = create_session_id()
        history = JsonChatHistory(session_id, directory=directory)
        try:
            with open(history.path, "x", encoding="utf-8") as f:
                # "x" 模式: 独占创建模式
                #   - 如果文件不存在, 创建新文件

```

```

# - 如果文件已存在，抛出 FileExistsError
# 这保证了并发安全：即使两个请求同时创建同名文件，
# 只有一个能成功，另一个会进入 except 重试
json.dump([], f, ensure_ascii=False)
# 写入空列表，表示这是一个空会话

except FileExistsError:
    # 文件已存在（极小概率的 ID 冲突），重新生成 ID 再试
    continue
return session_id
# 成功创建后返回会话 ID

```

```

def list_sessions(directory=None):
    """列出所有会话 ID，按时间倒序排列（最新的在前）。"""
    directory = directory or SESSIONS_DIR
    os.makedirs(directory, exist_ok=True)
    # 确保目录存在

    return sorted(
        (
            name[:-5]
            # 去掉文件名末尾的 ".json"（5 个字符）
            # "2026-09-10_20-08-13-139187_3bed07b9.json" → "2026-09-10_20-08-13-139187_3bed07b9"

            for name in os.listdir(directory)
            # os.listdir(directory): 列出目录下所有文件和文件夹名

            if name.endswith(".json") and not name.endswith(".meta.json")
            # 过滤条件：
            # - 必须以 .json 结尾（是 JSON 文件）
            # - 但不能以 .meta.json 结尾（排除设置文件，只取消息文件）
        ),
        reverse=True,
        # reverse=True: 降序排列，最新的在前
        # 因为会话 ID 以时间戳开头，按字符串排序就是按时间排序
    )

```

```

def delete_session(session_id, directory=None):
    """删除会话文件（包括消息文件和设置文件）。"""
    directory = directory or SESSIONS_DIR
    history = JsonChatHistory(session_id, directory=directory)
    if os.path.exists(history.path):
        os.remove(history.path)
        # 删除消息文件: {session_id}.json

    meta_path = os.path.join(directory, f"{session_id}.meta.json")
    if os.path.exists(meta_path):
        os.remove(meta_path)
        # 删除设置文件: {session_id}.meta.json
        # 设置文件存的是科目、性别、性格等配置

```

```

def save_session_meta(session_id, meta, directory=None):
    """把科目/性别/性格等设置持久化到 {session_id}.meta.json。"""
    directory = directory or SESSIONS_DIR
    os.makedirs(directory, exist_ok=True)
    path = os.path.join(directory, f"{session_id}.meta.json")

```

```

with open(path, "w", encoding="utf-8") as f:
    json.dump(meta, f, ensure_ascii=False)
# meta 是一个字典，如：
# {"subject": "数学", "gender": "女", "personality": "温柔耐心"}

```

```

def load_session_meta(session_id, directory=None):
    """读取 {session_id}.meta.json；文件不存在或损坏时返回空字典。"""
    directory = directory or SESSIONS_DIR
    path = os.path.join(directory, f"{session_id}.meta.json")
    if not os.path.exists(path):
        return {}
    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except (json.JSONDecodeError, OSError):
        # 文件损坏或无法读取，返回空字典
        return {}

```

```

def get_session_history(session_id):
    """工厂函数：返回指定 session_id 对应的 JsonChatHistory 实例。

    供 RunnableWithMessageHistory(get_session_history=...) 使用。
    """
    return JsonChatHistory(session_id)
# 这个函数看起来很简单，但它是 LangChain 记忆体系的关键入口。
# 当 RunnableWithMessageHistory 需要读取或写入历史时，
# 它会调用 get_session_history(session_id) 来获取对应的历史存储对象。

```

7.2 记忆数据流详解

用户说 "什么是函数？"



RunnableWithMessageHistory 拦截



① 从 config 中取出 session_id (如 "2026-09-10...")

② 调用 get_session_history(session_id)

└─ 返回 JsonChatHistory 实例

③ 调 .messages 属性读取历史

└─ 从 sessions/{session_id}.json 读取

└─ 用 messages_from_dict() 转成消息对象

④ 把历史消息填进 ChatPromptTemplate 的 {history} 插槽

⑤ 组装完整 prompt → 发给 LLM → 拿到回复

⑥ 调 .add_messages() 把本轮消息追加到 JSON 文件

└─ 读旧消息 + 追加新消息 → messages_to_dict() → json.dump()

7.3 为什么用 JSON 而不是数据库？

存储方式	优点	缺点
JSON 文件	简单、直观、不需要额外依赖、方便调试（直接打开看）	不适合高并发、大量数据时性能差
SQLite	支持复杂查询、并发性好	需要额外依赖、调试不方便
Redis	极快、支持过期	需要额外服务、数据在内存中

对于这个教学项目，JSON 文件是最合适的选择：简单直观，而且是本地单用户使用。

8. 模块五：会话管理 —— core/session_manager.py

新手先理解：上一个模块实现了"单个会话的读写"，但用户可能需要多个会话（比如同时学数学和英语）。这个模块负责管理所有会话的创建、切换、删除、清空，以及记住每个会话的"科目、性别、性格"设置。

8.1 完整源码与逐行注释

```
"""AI 教师会话管理。

这个类只负责会话状态和会话文件操作；具体的 Streamlit 页面控件仍由主程序渲染。
"""

import streamlit as st
# streamlit: Web 界面框架
# 这里用 st.session_state 来管理跨页面刷新的状态

from core.memory import (
    create_session,
    delete_session,
    get_session_history,
    list_sessions,
    load_session_meta,
    save_session_meta,
)
# 从 memory 模块导入所有需要的辅助函数
```

```

class SessionManager:
    """统一管理当前会话以及 sessions 目录中的会话。"""

    # — 类属性: Streamlit session_state 中的键名（常量） —
    CURRENT_KEY = "current_session"
    # 当前活动会话的 ID，存在 st.session_state["current_session"] 中

    PICKER_KEY = "session_picker"
    # 侧边栏下拉框的值，与 CURRENT_KEY 保持同步

    SUBJECT_KEY = "_subject_widget"
    # 科目下拉框的值，以 _ 开头表示这是内部使用的键

    GENDER_KEY = "_gender_widget"
    # 性别下拉框的值

    PERSONALITY_KEY = "_personality_widget"
    # 性格输入框的值

    def __init__(self, state=None):
        """初始化会话管理器。

        Args:
            state: Streamlit 的 session_state 对象。
                   默认用 st.session_state，测试时可以传入自定义字典。
        """
        self.state = state if state is not None else st.session_state
        # st.session_state 是 Streamlit 的"全局状态字典"
        # 它的特殊之处：页面刷新时，里面的值不会丢失
        # 普通变量在每次页面交互时都会重新初始化，但 session_state 中的值会保留

        self.ensure_current_session()
        # 初始化时自动加载最近的会话

    # — 属性（Property） —

    @property
    def current_session(self):
        """返回当前活动会话的 ID，没有则返回 None。"""
        return self.state.get(self.CURRENT_KEY)
        # .get() 安全读取，键不存在时返回 None

    @property
    def sessions(self):
        """返回所有会话 ID 列表（从磁盘读取，按时间倒序）。"""
        return list_sessions()
        # 每次访问都从磁盘重新读取，确保列表是最新的

    # — 初始化 —

    def ensure_current_session(self):
        """启动时优先加载最近的已有会话；没有会话时保持空白（None），不自动创建。

        此方法在 __init__ 中调用，早于任何 widget 渲染，可安全写 session_state。
        """
        if self.CURRENT_KEY not in self.state:
            # 首次运行（session_state 中还没有 current_session）
            existing = list_sessions()

```

```

    if existing:
        # 有已有会话，加载最近的一个
        self.state[self.CURRENT_KEY] = existing[0]
        # existing[0] 是最新的（list_sessions 按时间降序排列）
        self._apply_session_settings(existing[0])
        # 从磁盘加载该会话的科目/性别/性格设置
    else:
        # 没有任何会话，设为 None（空白状态）
        self.state[self.CURRENT_KEY] = None

# 同步下拉框的初始值
self.state[self.PICKER_KEY] = self.state.get(self.CURRENT_KEY)

# — 设置持久化（磁盘 .meta.json） —

def _apply_session_settings(self, session_id):
    """从磁盘 meta 文件还原 widget 值到 session_state。

    在 on_click/on_change 回调或渲染前调用。
    widget 渲染时会自动从 session_state 读取对应的 key 作为默认值。
    """
    if session_id is None:
        # 没有会话（空白状态），重置为默认值
        if self.SUBJECT_KEY in self.state:
            del self.state[self.SUBJECT_KEY]
            # 删除科目键，让主程序感知到需要补默认值
        self.state[self.GENDER_KEY] = "男"
        self.state[self.PERSONALITY_KEY] = ""
        return

    meta = load_session_meta(session_id)
    # 从 {session_id}.meta.json 读取设置

    if meta.get("subject") is not None:
        self.state[self.SUBJECT_KEY] = meta["subject"]
    elif self.SUBJECT_KEY in self.state:
        del self.state[self.SUBJECT_KEY]
        # 如果 meta 中没有科目，删除键让主程序补默认值

    self.state[self.GENDER_KEY] = meta.get("gender", "男")
    # .get("gender", "男")：如果 meta 中没有 gender，默认"男"

    self.state[self.PERSONALITY_KEY] = meta.get("personality", "")
    # .get("personality", "")：如果 meta 中没有 personality，默认空字符串

def _save_current_settings(self):
    """把当前 widget 值写到当前会话的磁盘 meta 文件。切换/删除前调用。"""
    sid = self.current_session
    if not sid:
        return
    save_session_meta(sid, {
        "subject": self.state.get(self.SUBJECT_KEY),
        "gender": self.state.get(self.GENDER_KEY, "男"),
        "personality": self.state.get(self.PERSONALITY_KEY, ""),
    })

def save_session_settings(self, subject, gender, personality):
    """由主程序在用户发送消息时调用，确保不切换时也能持久化设置。"""
    sid = self.current_session

```

```

if not sid:
    return
save_session_meta(sid, {
    "subject": subject,
    "gender": gender,
    "personality": personality,
})

```

— 会话操作（作为 on_click / on_change 回调使用） —

```
def new_session(self) -> None:
```

```
    """创建并切换到新会话。
```

优先复用已存在的最新空会话（无消息），避免重复创建空文件。

作为按钮 on_click 回调使用，执行时 widget 尚未实例化，可安全写 PICKER_KEY。

```
    """
```

```
    # 找最近创建的空会话（list_sessions 按时间降序排列）
```

```
    for sid in self.sessions:
```

```
        if not get_session_history(sid).messages:
```

```
            # 这个会话的 messages 为空列表 → 是空会话，可以复用
```

```
            if sid != self.current_session:
```

```
                # 不是当前会话，切换过去
```

```
                self._save_current_settings()
```

```
                # 先把当前会话的设置保存到磁盘
```

```
                self.state[self.CURRENT_KEY] = sid
```

```
                self.state[self.PICKER_KEY] = sid
```

```
                self._apply_session_settings(sid)
```

```
                # 加载新会话的设置
```

```
            return
```

```
    # 所有会话都有内容，才真正新建
```

```
    self._save_current_settings()
```

```
    session_id = create_session()
```

```
    # 创建新的空会话文件
```

```
    self.state[self.CURRENT_KEY] = session_id
```

```
    self.state[self.PICKER_KEY] = session_id
```

```
    self._apply_session_settings(session_id)
```

```
def load_session(self, session_id):
```

```
    """切换到指定会话。不写 PICKER_KEY，避免在 on_change 回调中触发 widget 错误。"""
```

```
    if session_id and session_id in self.sessions and session_id != self.current_session:
```

```
        self._save_current_settings()
```

```
        self.state[self.CURRENT_KEY] = session_id
```

```
        self._apply_session_settings(session_id)
```

```
    return self.current_session
```

```
def load_selected_session(self) -> None:
```

```
    """selectbox on_change 回调：读取下拉框当前值并切换会话。"""
```

```
    self.load_session(self.state.get(self.PICKER_KEY))
```

```
def delete_current_session(self):
```

```
    """删除当前会话，切换到下一个已有会话；没有则进入空白状态（None）。"""
```

作为按钮 on_click 回调使用，执行时 widget 尚未实例化，可安全写 PICKER_KEY。

```
    """
```

```
    sid = self.current_session
```

```
    if not sid:
```

```

        return
    delete_session(sid)
    # 删除消息文件和 meta 文件

    remaining = self.sessions
    # 重新列出剩余会话（已排除刚删除的）

    next_id = remaining[0] if remaining else None
    # 有剩余会话就切到第一个，没有就 None

    self.state[self.CURRENT_KEY] = next_id
    self.state[self.PICKER_KEY] = next_id
    self._apply_session_settings(next_id)

def clear_current_session(self):
    """清空当前会话消息，但保留会话文件和 ID。"""
    if self.current_session:
        get_session_history(self.current_session).clear()
        # 调用 JsonChatHistory.clear()，把消息文件内容清空为 []

def get_history(self):
    """返回当前会话的 LangChain 历史对象；无当前会话时返回 None。"""
    if not self.current_session:
        return None
    return get_session_history(self.current_session)

```

8.2 会话生命周期



9. 模块六：RAG 检索（占位） —— core/rag.py

新手先理解：RAG（Retrieval-Augmented Generation，检索增强生成）是让 AI 在回答前先"查资料"的技术。目前这个模块只是占位，计划在 Phase 3 实现。

"""

RAG 检索模块（占位，最后阶段 Phase 3 再实现）

计划流程：

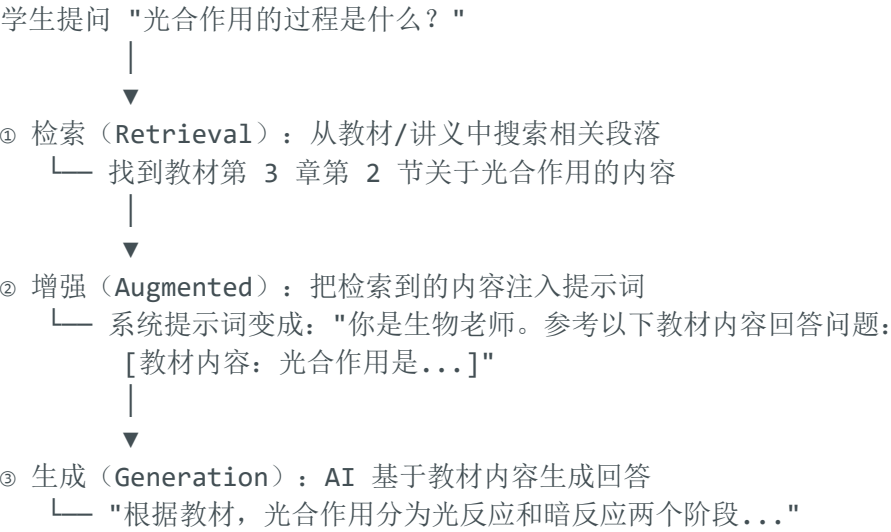
- 加载 data/ 下的教材/讲义（PDF、Word、Markdown）
- > 用 RecursiveCharacterTextSplitter 切片
- > 向量化（Embedding，需另选模型，DeepSeek 无公开 Embedding API）
- > 入库（Chroma 或 FAISS）
- > as_retriever() 检索相关知识，注入提示词

"""

TODO(Phase 3)：实现文档加载、切片、向量化、检索器与 RAG 链

9.1 RAG 是什么？（给新人的解释）

想象一个场景：学生问"光合作用的过程是什么？"，如果只靠 AI 自身的知识，回答可能不准确或过时。RAG 的做法是：



9.2 RAG 的技术栈（计划）

步骤	使用的工具	说明
文档加载	langchain_community.document_loaders	加载 PDF、Word、Markdown 等格式

步骤	使用的工具	说明
文档切片	<code>RecursiveCharacterTextSplitter</code>	把长文档切成小段（每段 500-1000 字）
向量化	Embedding 模型（如 <code>text2vec</code> 、 <code>bge</code> 等）	把文本转成数学向量（用于相似度搜索）
向量库	Chroma 或 FAISS	存储向量，支持快速相似度搜索
检索	<code>as_retriever()</code>	根据问题检索最相关的文档片段
注入	在 <code>build_chat_prompt()</code> 中加入检索结果	把检索到的内容追加到 system prompt 中

10. 模块七：主应用 —— `ai_teacher_app.py`

新手先理解：这是整个项目的"指挥中心"，把所有模块串起来，并提供一个 Web 聊天界面。这里你会看到 LangChain 最核心的用法：**链式调用**。

10.1 完整源码与逐行注释

```
"""AI 智能教师 Streamlit 应用（Phase 2）。"""

# =====
# 第一部分：导入依赖
# =====

import streamlit as st
# streamlit: Web 界面框架
# 只需写 Python 代码，自动生成网页界面
# 核心概念：每次用户交互（点击、输入），整个脚本从上到下重新执行一遍

from langchain_core.runnables.history import RunnableWithMessageHistory
# RunnableWithMessageHistory: LangChain 的记忆包装器
# 它把普通的 chain 包装成"有记忆的 chain"
# 调用时自动：读历史 → 填模板 → 调模型 → 写历史

from core.llm import get_llm
# 获取封装好的 DeepSeek 模型实例

from core.memory import get_session_history
```

```
# 获取会话历史的工厂函数
```

```
from core.prompt_builder import build_chat_prompt
# 构建 ChatPromptTemplate 的函数
```

```
from prompts.prompt_manager import PromptManager
# 提示词库管理器
```

```
from core.session_manager import SessionManager
# 会话管理器
```

```
# =====
# 第二部分：页面配置
# =====
```

```
st.set_page_config(
    page_title="AI智能教师",    # 浏览器标签页标题
    page_icon="👨🏫",           # 标签页图标 (emoji)
    layout="wide"               # 宽屏布局 (而不是窄列居中)
)
# 这个函数必须是 Streamlit 脚本的第一个 st 调用
# 放在最前面才能生效
```

```
pm = PromptManager()
# 创建提示词管理器 (全局单例)
# 它会在初始化时加载 JSON 文件
```

```
session_manager = SessionManager()
# 创建会话管理器 (全局单例)
# 它会在初始化时自动加载最近的会话
```

```
# =====
# 第三部分：核心函数
# =====
```

```
def build_chain(subject, gender, personality):
    """构建带历史记录的对话链。
```

先根据科目、性别和性格构建聊天提示模板，再与 LLM 组合成可运行链，最后通过 RunnableWithMessageHistory 包装，使链具备对话历史记忆能力。

Args:

subject: 科目名称，如 "数学"、"语文" 等
gender: 教师性别，"男" 或 "女"
personality: 教师性格描述，如 "温柔耐心、擅长用生活例子讲解"

Returns:

RunnableWithMessageHistory: 包装了消息历史的可运行链对象

"""

— 步骤 1: 构建基本链 —

```
chain = build_chat_prompt(subject, gender, personality) | get_llm()
```

拆解这个表达式:

#

```
# build_chat_prompt(subject, gender, personality)
```

└─ 返回 ChatPromptTemplate (含 system + history + human 三段)

#

| ← 这是 LangChain 的 LCEL 管道运算符 (pipe)

```

# — 意思是：把左边（prompt）的输出，作为右边（llm）的输入
#
# get_llm()
# — 返回 ChatOpenAI 实例（DeepSeek 模型）
#
# 整条链的数据流：
# 用户输入 → ChatPromptTemplate 格式化 → ChatOpenAI 生成 → 模型回复
# chain 本身也是一个 Runnable 对象，可以 .invoke() 或 .stream()

# — 步骤 2：包装记忆 —
return RunnableWithMessageHistory(
    chain,
    # 要包装的基础链（prompt | llm）

    get_session_history,
    # 工厂函数：根据 session_id 获取历史存储对象
    # 当链被调用时，LangChain 会：
    # 1. 从 config 中取出 session_id
    # 2. 调用 get_session_history(session_id) 获取历史
    # 3. 把历史填进 ChatPromptTemplate 的 {history} 插槽

    input_messages_key="input",
    # 指定输入字典中哪个键是用户的新消息
    # 链被调用时传 {"input": "你好"}，这个参数告诉 LangChain:
    # "请把 'input' 键对应的值作为用户的新消息"

    history_messages_key="history",
    # 指定 ChatPromptTemplate 中历史消息的插槽名
    # 对应 ChatPromptTemplate 中的 MessagesPlaceholder(variable_name="history")
    # 这两个名字必须一致！
)

```

```

def extract_text(chunk):
    """从 LLM 流式输出的 chunk 中提取文本内容。

    兼容多种 chunk 格式：
    - 带 content 属性的对象（如 AIMessageChunk）
    - 直接传入的字符串
    - 包含多个 dict 的列表（如 OpenAI 格式的 choices）

    Args:
        chunk: LLM 流式输出的单个数据块，支持多种类型

    Returns:
        str: 提取出的文本字符串，若 chunk 为 None 则返回空字符串
    """
    content = getattr(chunk, "content", chunk)
    # getattr(obj, "attr_name", default):
    # - 如果 chunk 有 content 属性，返回 chunk.content
    # - 如果 chunk 没有 content 属性（比如它本身就是字符串），返回 chunk 本身
    # 这样就能兼容多种 chunk 格式

    if content is None:
        return ""
        # content 为 None: 比如 DeepSeek 的思考阶段，流式返回 delta.content=None
        # 此时返回空字符串，跳过这个 chunk

    if isinstance(content, str):

```

```
return content
# 是最常见的情况: content 就是一段文本
# 比如 chunk.content = "你好"
```

```
if isinstance(content, list):
    # 某些模型（如原生 OpenAI 格式）返回的 content 是一个列表
    # 格式: [{"text": "你好", "type": "text"}, ...]
    return "".join(
        x.get("text", "") if isinstance(x, dict) else str(x)
        for x in content
    )
    # 遍历列表, 取出每个元素的 "text" 字段, 拼接成字符串
    # .get("text", ""): 如果字典中没有 "text" 键, 返回空字符串

return str(content)
# 最后的兜底: 把 content 转成字符串
```

```
# =====
# 第四部分: 侧边栏 UI
# =====
```

```
with st.sidebar:
    # st.sidebar 是一个上下文管理器, 里面的所有组件都会渲染到侧边栏
    # 就像写一个 with 块, 里面的内容自动放到侧边栏

    st.subheader("AI 教师设置")
    # 侧边栏标题

    # — 新建会话按钮 —
    st.button(
        "🛠️ 新建会话",
        # 按钮上显示的文字

        use_container_width=True,
        # 按钮宽度撑满侧边栏

        on_click=session_manager.new_session
        # on_click 参数: 点击按钮时执行的回调函数
        # 注意: 这里传的是函数本身 (session_manager.new_session), 不是调用结果
        # 不要写成 on_click=session_manager.new_session() ← 这是错误的!
        # 关键: on_click 回调在 widget 渲染前执行,
        # 避免 StreamlitWidgetAlreadyInstantiatedError (重复实例化错误)
    )

    # — 会话列表 —
    sessions = session_manager.sessions
    # 从磁盘获取所有会话 ID 列表

    if sessions:
        # 有会话时才显示加载和删除功能

        current = session_manager.current_session
        # 当前活动的会话 ID

        index = sessions.index(current) if current in sessions else 0
        # 计算下拉框的默认选中位置
        # - 如果当前会话在列表中, 找到它的索引
        # - 如果不在 (被删除后的短暂状态), 默认选第一个 (索引 0)
```

```

st.selectbox(
    "加载会话",
    # 下拉框标签

    sessions,
    # 下拉框选项列表

    index=index,
    # 默认选中的选项索引

    key=session_manager.PICKER_KEY,
    # key 参数: 把这个下拉框的值绑定到 session_state 的指定键
    # 这样 SessionManager 就能通过 session_state 读取用户的选择

    on_change=session_manager.load_selected_session
    # on_change 参数: 当用户切换下拉框选项时执行的回调
)

st.button(
    "🗑️ 删除当前会话",
    use_container_width=True,
    on_click=session_manager.delete_current_session
    # 点击后删除当前会话, 自动切换到下一个
)

# — 科目选择 —
subjects = pm.list_subjects()
# 获取所有科目名列表: ["语文", "数学", "英语", ...]

if session_manager.SUBJECT_KEY not in st.session_state:
    # 科目键不存在: 说明刚切换到新会话, 需要补入默认值
    st.session_state[session_manager.SUBJECT_KEY] = subjects[0]
    # 默认选第一个科目

subject = st.selectbox(
    "科目",
    # 标签

    subjects,
    # 选项: 9 个科目

    key=session_manager.SUBJECT_KEY
    # 绑定到 session_state, 方便 SessionManager 读写
)

# — 性别选择 —
gender = st.selectbox(
    "性别",
    ["男", "女"],
    key=session_manager.GENDER_KEY
)

# — 性格输入 —
personality = st.text_input(
    "性格",
    placeholder="如: 温柔耐心、擅长用生活例子讲解",
    # placeholder: 输入框为空时显示的提示文字 (灰色)
    key=session_manager.PERSONALITY_KEY
)

```

```

)

# — 清空会话按钮 —
if st.button("🗑️ 清空当前会话", use_container_width=True):
    session_manager.clear_current_session()
    # 清空消息（保留会话文件和 ID）
    st.rerun()
    # st.rerun(): 强制重新运行整个脚本
    # 清空后需要刷新页面，让聊天记录消失

# =====
# 第五部分：主页面内容
# =====

st.title("👩‍🏫 AI 智能教师")
# 页面大标题

current_session = session_manager.current_session
# 获取当前会话 ID

if not current_session:
    # 空白态：还没有任何会话
    st.info("点击左侧「+ 新建会话」开始对话。")
    # st.info(): 显示蓝色信息提示框
    st.stop()
    # st.stop(): 立即停止脚本执行，后面的代码不会运行
    # 这是 Streamlit 的控制流方式：条件不满足就提前终止

# — 显示当前会话信息 —
st.caption(
    f"当前会话: {current_session} · {subject}教师 · {gender} · {personality or '未设置性格'}"
)
# st.caption(): 显示灰色小字说明
# personality or '未设置性格': 如果 personality 为空字符串，显示"未设置性格"

# — 渲染历史消息 —
history = session_manager.get_history()
# 获取当前会话的 JsonChatHistory 对象

messages = []
if history is not None:
    try:
        messages = history.messages
        # 调用 @property messages，从磁盘读取历史消息
    except ValueError as exc:
        st.error(str(exc))
        # 如果 JSON 文件损坏，显示错误信息

for msg in messages:
    # 遍历历史消息
    st.chat_message(
        "user" if msg.type == "human" else "assistant"
        # st.chat_message() 的参数是角色名：
        # - "user": 显示用户头像（右侧聊天气泡）
        # - "assistant": 显示 AI 头像（左侧聊天气泡）
        # msg.type 是 LangChain 消息对象的类型属性：
        # - "human" 表示用户消息

```

```
        # - "ai" 表示 AI 回复
    ).write(msg.content)
# .write(): 在聊天气泡中显示消息内容
```

```
# =====
# 第六部分：聊天输入与回复
# =====
```

```
user_prompt = st.chat_input("请输入你的问题")
# st.chat_input(): 聊天输入框（固定在页面底部）
# 返回用户输入的文本，如果用户还没输入则返回 None
```

```
if user_prompt:
    # 用户发送了消息
```

```
    # — 校验：必须填写完整设置 —
```

```
    if not (subject and gender and personality.strip()):
        # personality.strip(): 去掉首尾空格
        # 如果 personality 全是空格，strip() 后变成空字符串
        st.error("请先在左侧填写：科目、性别、性格")
        st.stop()
```

```
    # — 持久化设置 —
```

```
    session_manager.save_session_settings(subject, gender, personality)
    # 把当前的科目、性别、性格保存到 .meta.json 文件
    # 这样下次加载这个会话时，设置会自动恢复
```

```
    # — 显示用户消息 —
```

```
    st.chat_message("user").write(user_prompt)
```

```
    # — 生成 AI 回复 —
```

```
    with st.chat_message("assistant"):
        # 创建一个 AI 聊天气泡
```

```
        placeholder = st.empty()
        # st.empty(): 创建一个空的占位符容器
        # 后续可以往里面放内容，每次放都会替换之前的内容
        # 这里用来实现"打字机效果"：不断更新同一个位置
```

```
        full_response = ""
        # 累积完整回复的变量
```

```
        try:
```

```
            chain = build_chain(subject, gender, personality)
            # 构建带记忆的对话链
```

```
            config = {"configurable": {"session_id": current_session}}
            # config 是传给 RunnableWithMessageHistory 的配置字典
            # 结构固定为 {"configurable": {"session_id": "..."}}
            # LangChain 通过这个字典来识别"当前是哪个会话"
            # 不同会话用不同的 session_id，历史就不会混淆
```

```
            for chunk in chain.stream(
                {"input": user_prompt},
                # 输入字典：键 "input" 对应 ChatPromptTemplate 中的 {input}
                # 值 user_prompt 就是用户在输入框中输入的文字
```

```
                config=config
```

```

# 传入包含 session_id 的配置，让记忆组件知道是哪个会话
):
    # .stream() 是流式调用方法
    # 每次迭代返回一个 chunk（数据块），每个 chunk 包含模型刚生成的一小段文
    本

    text = extract_text(chunk)
    # 从 chunk 中提取文本内容

    if text:
        full_response += text
        # 累积文本

        placeholder.markdown(full_response)
        # 用 markdown 格式渲染累积的文本
        # 每次循环都更新同一个 placeholder，实现逐字显示的效果

    if not full_response:
        # 模型没有返回任何文本内容
        st.warning("模型没有返回文本内容，本次消息未写入历史。")
        # st.warning(): 显示黄色警告框

except (EnvironmentError, ValueError) as exc:
    # EnvironmentError: API Key 没配置
    # ValueError: JSON 文件损坏等
    st.error(str(exc))
    # st.error(): 显示红色错误框

except Exception as exc:
    # 其他未预期的错误（网络问题、服务故障等）
    st.error(f"调用模型失败，请检查网络或服务状态：{exc}")

```

10.2 Streamlit 的执行模型（新人必读）

理解 Streamlit 的关键：**每次用户交互，整个脚本从头到尾重新执行。**



但 `st.session_state` 中的值**不会丢失**！这就是为什么 `SessionManager` 把状态存在 `st.session_state` 中——页面刷新后，会话 ID、科目、性别、性格等设置都还在。

10.3 流式输出（打字机效果）原理

`chain.stream()` 返回一个迭代器



`placeholder.markdown()` 每次调用都会**替换**之前的内容，所以看起来就像文字在逐字出现。

11. 核心技术点深入讲解

11.1 LCEL 管道运算符 |

这是 LangChain 最重要的概念之一。| 是 LangChain Expression Language (LCEL) 的核心：

```
# 基本用法
chain = prompt | llm
# 等价于: chain = RunnableSequence(prompt, llm)
# 数据流: 输入 → prompt → llm → 输出

# 可以串联更多组件
chain = prompt | llm | output_parser
# 数据流: 输入 → prompt → llm → output_parser → 输出
```

为什么用 | 而不是函数调用？

```
# ❌ 传统方式（嵌套函数调用）
def chat(input_text):
```

```
formatted = prompt.format(input_text)
response = llm(formatted)
parsed = parser(response)
return parsed
```

```
# ✅ LangChain 方式（管道链）
chain = prompt | llm | parser
response = chain.invoke({"input": input_text})
```

管道链的优势：

- **可读性强**：从左到右读，一目了然
- **可组合**：组件可以任意拆分和重组
- **统一接口**：所有组件都有 `.invoke()` 和 `.stream()` 方法

11.2 RunnableWithMessageHistory 的工作原理

这是 LangChain 记忆体系的核心。它的工作流程：

```
chain_with_history = RunnableWithMessageHistory(
    chain,                    # 基础链: prompt | llm
    get_session_history,      # 工厂函数: 根据 session_id 获取历史
    input_messages_key="input",
    history_messages_key="history",
)

# 调用时:
chain_with_history.invoke(
    {"input": "什么是函数? "},
    config={"configurable": {"session_id": "abc123"}}
)

# 内部执行流程:
# ① 从 config 中取出 session_id = "abc123"
# ② 调用 get_session_history("abc123") → 返回 JsonChatHistory 实例
# ③ 调 .messages 属性读取历史 → [HumanMessage("你好"), AIMessage("你好!")]
# ④ 把历史填入 ChatPromptTemplate 的 {history} 位置
# ⑤ 组装完整消息: [system, ...历史消息, human("什么是函数?")]
# ⑥ 发给 LLM → 拿到回复
# ⑦ 调 .add_messages([HumanMessage("什么是函数?"), AIMessage("函数是...")])
# ⑧ 返回 LLM 的回复
```

11.3 ChatPromptTemplate 的三段式结构

```
ChatPromptTemplate.from_messages([
    ("system", "你是数学老师..."), # ① 系统指令：定义 AI 的角色和行为
    MessagesPlaceholder("history"), # ② 历史消息：之前的对话记录
    ("human", "{input}"), # ③ 用户输入：当前问题
])
```

最终发给 LLM 的消息格式（OpenAI 兼容格式）：

```
[
    {"role": "system", "content": "你是数学老师，性别：女，教学性格：温柔耐心..."},
    {"role": "user", "content": "什么是函数？"},
    {"role": "assistant", "content": "函数是一种映射关系..."},
    {"role": "user", "content": "能举个例子吗？"}
]
```

11.4 BaseChatMessageHistory 的接口约定

LangChain 1.x 的 `BaseChatMessageHistory` 要求子类实现：

方法/属性	类型	作用
<code>messages</code>	property	返回所有消息列表（从存储中读取）
<code>add_messages(messages)</code>	方法	批量追加消息（写入存储）
<code>clear()</code>	方法	清空所有消息

注意：`add_message`（单条）不需要实现，基类会自动委托给 `add_messages`。

11.5 messages_to_dict / messages_from_dict

这两个函数是 LangChain 官方的消息序列化工具：

```
from langchain_core.messages import messages_from_dict, messages_to_dict,
HumanMessage, AIMessage

# 消息对象 → 字典（序列化，用于存盘）
messages = [HumanMessage(content="你好"), AIMessage(content="你好！")]
dicts = messages_to_dict(messages)
# 结果: [{"type": "human", "content": "你好"}, {"type": "ai", "content": "你好！"}]

# 字典 → 消息对象（反序列化，用于读取）
```

```
restored = messages_from_dict(dicts)
# 结果: [HumanMessage(content="你好"), AIMessage(content="你好! ")]
```

11.6 Streamlit session_state 的作用

```
# 普通变量: 每次页面交互都会重新初始化
count = 0 # 每次都是 0!

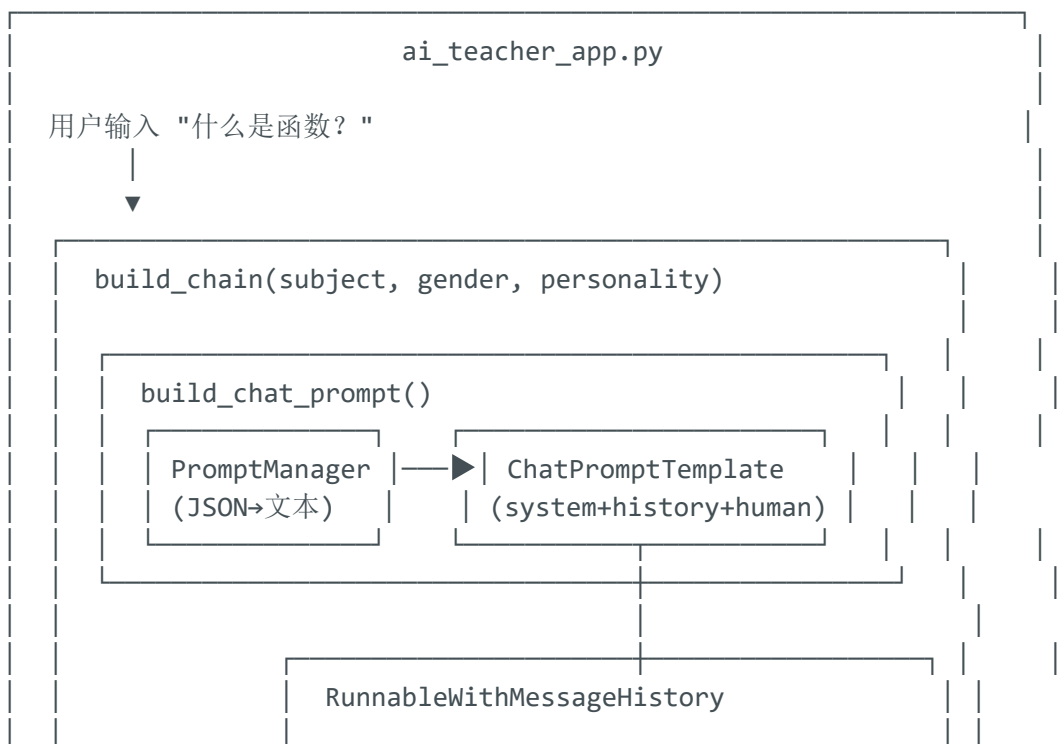
# session_state: 值在页面刷新后保留
if "count" not in st.session_state:
    st.session_state.count = 0 # 只在第一次运行时初始化
st.session_state.count += 1    # 每次点击都会累加
```

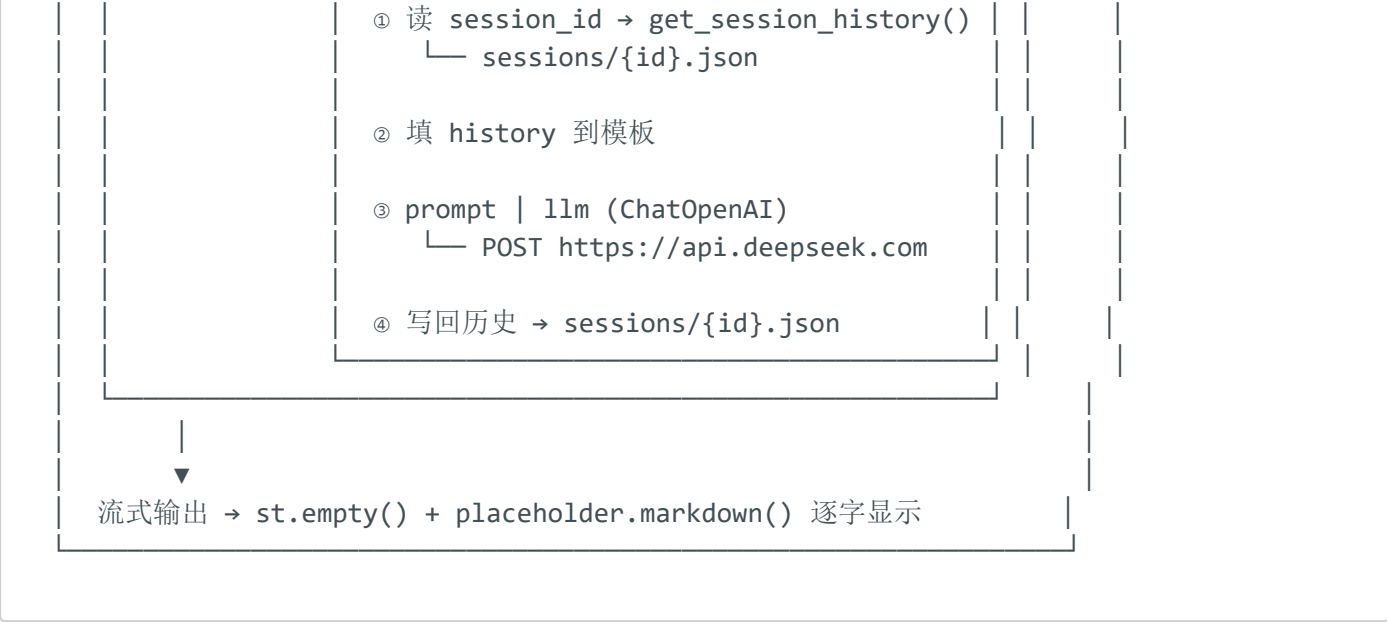
本项目用 `session_state` 存储:

- `current_session`: 当前会话 ID
- `session_picker`: 下拉框的值
- `_subject_widget` / `_gender_widget` / `_personality_widget`: 用户的设置

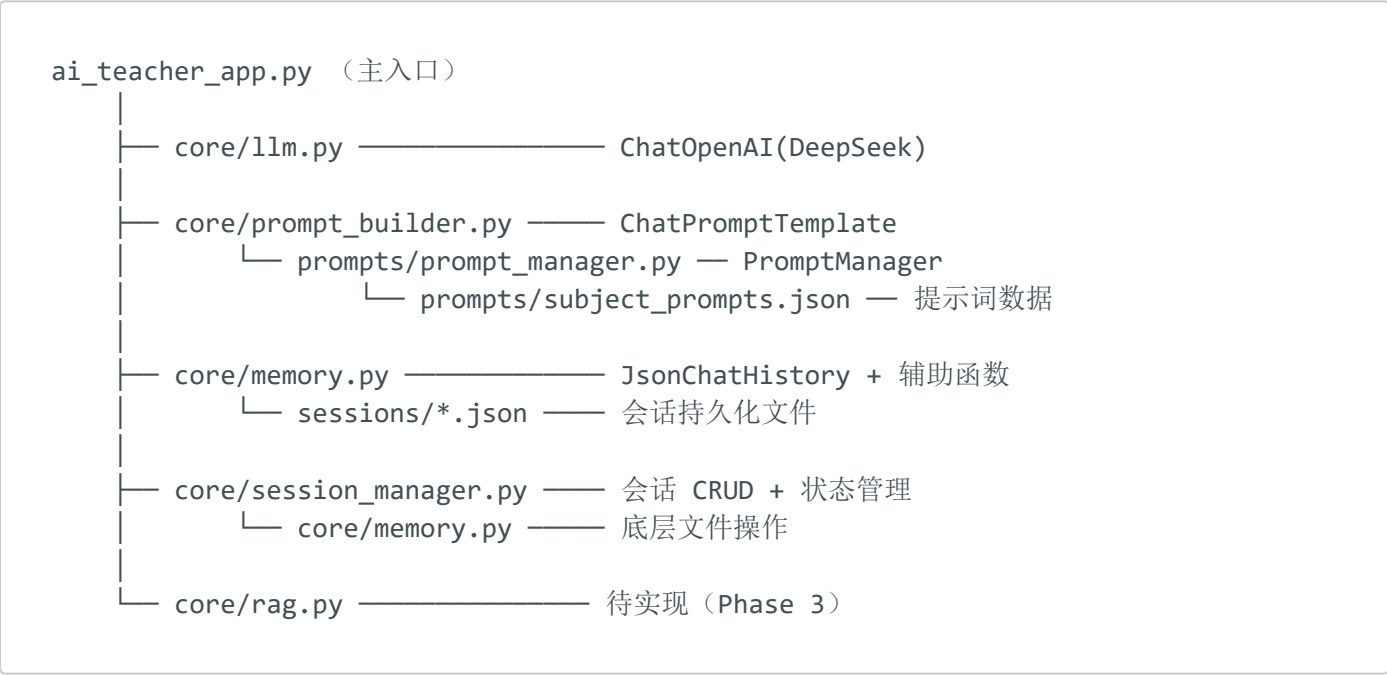
12. 数据流全景图

12.1 一次完整对话的数据流





12.2 文件之间的依赖关系



12.3 各模块职责总结

模块	文件	一句话职责
模型封装	core/llm.py	统一管理 DeepSeek 模型的参数和配置
提示词数据	prompts/subject_prompts.json	9 科提示词的数据存储
提示词逻辑	prompts/prompt_manager.py	读取 JSON 并按变量拼接提示词

模块	文件	一句话职责
提示词模板	<code>core/prompt_builder.py</code>	把文本提示词包装成 LangChain 模板
记忆存储	<code>core/memory.py</code>	实现 JSON 文件持久化的会话记忆
会话管理	<code>core/session_manager.py</code>	管理会话的创建、切换、删除、设置
RAG 检索	<code>core/rag.py</code>	检索增强生成（待实现）
主应用	<code>ai_teacher_app.py</code>	Streamlit UI + 链组装 + 流式输出

13. 如何运行与调试

13.1 环境准备

```
# 1. 激活 conda 环境
conda activate langchain1.2

# 2. 确认已安装的包
pip list | findstr langchain
# 应该能看到: langchain, langchain-core, langchain-openai, streamlit 等

# 3. 设置 API Key（二选一）
# 方式 A: 临时设置（当前终端有效）
$env:DEEPSEEK_API_KEY="sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

# 方式 B: 永久设置（推荐）
# 在系统环境变量中添加 DEEPSEEK_API_KEY
```

13.2 启动应用

```
# 进入项目目录
cd D:\桌面\Studing\data-learning\AI_application\AI_Teacher

# 启动 Streamlit
streamlit run ai_teacher_app.py
```

启动后浏览器会自动打开 `http://localhost:8501`。

13.3 测试流程

- 1. 首次启动：页面显示"点击左侧「+ 新建会话」开始对话"
- 2. 新建会话：点击侧边栏的"+ 新建会话"
- 3. 设置参数：选择科目、性别，输入性格描述
- 4. 发送消息：在底部输入框输入问题，观察打字机效果
- 5. 查看历史：刷新页面，历史消息仍然存在
- 6. 切换会话：新建另一个会话，在下拉框中切换
- 7. 清空会话：点击"🗑️ 清空当前会话"
- 8. 删除会话：点击"🗑️ 删除当前会话"

13.4 常见问题排查

问题	原因	解决方法
ModuleNotFoundError: No module named 'langchain_openai'	环境没切对	检查 <code>conda activate langchain1.2</code>
未找到 DEEPSEEK_API_KEY	环境变量没设置	设置 <code>\$env:DEEPSEEK_API_KEY="sk-..."</code>
调用模型失败，请检查网络或服务状态	网络问题或 API 不可用	检查网络，确认 API Key 有效
会话文件损坏或无法读取	JSON 文件被手动修改损坏	删除 <code>sessions/</code> 下对应文件
下拉框内容不更新	Streamlit 缓存问题	按 <code>Ctrl+Shift+R</code> 强制刷新
流式输出显示乱码	编码问题	确认所有文件使用 UTF-8 编码

附录：LangChain 学习路线图

如果你是这个项目的新人，建议按以下顺序学习 LangChain：

第一阶段：理解核心概念

└─ 什么是 LLM（大语言模型）

- └─ 什么是 Prompt（提示词）
- └─ 什么是 Chain（链）
- └─ 什么是 Memory（记忆）

第二阶段：本项目实战

- └─ ① 先看 `llm.py` — 理解怎么封装模型
- └─ ② 再看 `prompt_manager.py` — 理解提示词管理
- └─ ③ 看 `prompt_builder.py` — 理解 ChatPromptTemplate
- └─ ④ 看 `memory.py` — 理解记忆持久化
- └─ ⑤ 看 `session_manager.py` — 理解会话管理
- └─ ⑥ 看 `ai_teacher_app.py` — 理解整体串联
- └─ ⑦ 动手修改：试着加一个新科目、改提示词

第三阶段：进阶

- └─ RAG（检索增强生成）
- └─ Agent（智能代理）
- └─ Tool Calling（工具调用）
- └─ LangGraph（状态图编排）

最后的话：这个项目虽然不大，但涵盖了 LangChain 项目落地的所有核心环节：模型封装、提示词管理、记忆持久化、会话管理、流式输出。把这些基础打牢，后面学 RAG、Agent、LangGraph 都会事半功倍。祝你学习愉快！🎓